

SQL Assignment & Practice Bank — 200 Questions

Basic → Intermediate: Questions 1–180 | Hard Challenges: Questions 181–200

Instructions: Write and execute SQL queries for each problem.

1. Database & Table Creation (1–20)

- Q1.** Create a database named `company_db`.
- Q2.** Select and use the `company_db` database.
- Q3.** Create an `employees` table with `employee_id`, `employee_name`, `department`, `salary`, `city`, and `hire_date` columns.
- Q4.** Make `employee_id` the PRIMARY KEY.
- Q5.** Make `employee_name` NOT NULL.
- Q6.** Add a UNIQUE constraint to an email column.
- Q7.** Add a CHECK constraint so salary must be greater than 0.
- Q8.** Add a DEFAULT value of 'Hyderabad' for the city column.
- Q9.** Create a `departments` table with `department_id`, `department_name`, `location`, and `budget`.
- Q10.** Make `department_id` the PRIMARY KEY in the `departments` table.
- Q11.** Create a `projects` table with `project_id`, `project_name`, `department_id`, and `budget`.
- Q12.** Add a FOREIGN KEY from `employees.department_id` to `departments.department_id`.
- Q13.** Insert one employee record.
- Q14.** Insert five employee records in a single INSERT statement.
- Q15.** Insert a department record.
- Q16.** Create a `students` table with PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.
- Q17.** Create a table named `attendance` with `employee_id`, `attendance_date`, and `status`.
- Q18.** Create a table only if it does not already exist.
- Q19.** Describe or display the structure of the `employees` table.
- Q20.** Drop a temporary table named `temp_data` if it exists.

2. Basic SELECT & Filtering (21–45)

- Q21.** Display all columns from the `employees` table.
- Q22.** Display only `employee_name` and `salary`.
- Q23.** Display employees who live in Hyderabad.
- Q24.** Display employees whose salary is greater than 60000.
- Q25.** Display employees whose salary is less than or equal to 50000.
- Q26.** Display employees whose salary is between 50000 and 70000.
- Q27.** Display employees who live in Hyderabad, Bangalore, or Mumbai using IN.
- Q28.** Display employees who do not live in Hyderabad.

- Q29. Display employees whose names start with 'A'.
- Q30. Display employees whose names end with 'a'.
- Q31. Display employees whose names contain the letter 'i'.
- Q32. Display employees whose department is NULL.
- Q33. Display employees whose department is NOT NULL.
- Q34. Display employees hired after 2022-01-01.
- Q35. Display employees hired between two given dates.
- Q36. Display employees with salaries greater than 50000 AND who live in Hyderabad.
- Q37. Display employees who live in Hyderabad OR earn more than 70000.
- Q38. Display employees who are NOT in the HR department.
- Q39. Display distinct cities from the employees table.
- Q40. Display distinct departments from the employees table.
- Q41. Display employees sorted by salary in ascending order.
- Q42. Display employees sorted by salary in descending order.
- Q43. Display employees sorted first by department and then by salary descending.
- Q44. Display the first 5 employees after sorting by employee_id.
- Q45. Display the top 3 highest-paid employees.

3. UPDATE, DELETE, ALTER & Basic Functions (46–65)

- Q46. Update the salary of one employee using employee_id.
- Q47. Increase the salary of all employees by 10 percent.
- Q48. Increase the salary of employees in the Data Science department by 5000.
- Q49. Change the city of all employees currently living in Bangalore to Bengaluru.
- Q50. Delete one employee using employee_id.
- Q51. Delete employees whose salary is below a specified amount.
- Q52. Add an email column to the employees table.
- Q53. Add a phone_number column to the employees table.
- Q54. Modify the length of the employee_name column.
- Q55. Rename the city column to employee_city.
- Q56. Drop the phone_number column.
- Q57. Display employee names in uppercase.
- Q58. Display employee names in lowercase.
- Q59. Display the length of every employee name.
- Q60. Display the first three characters of every employee name.
- Q61. Display employee_name and salary rounded to the nearest thousand using an appropriate expression.
- Q62. Display the current date.

- Q63.** Calculate how many years each employee has worked using hire_date.
- Q64.** Replace NULL department values with 'Unassigned' in the query result.
- Q65.** Use CASE to label employees as Low, Medium, or High salary based on salary ranges.

4. Aggregate Functions, GROUP BY & HAVING (66–90)

- Q66.** Find the total number of employees.
- Q67.** Find the total salary paid to all employees.
- Q68.** Find the average salary of all employees.
- Q69.** Find the highest salary.
- Q70.** Find the lowest salary.
- Q71.** Find the number of employees in Hyderabad.
- Q72.** Find the total salary paid to employees in Mumbai.
- Q73.** Find the average salary of employees in Bangalore.
- Q74.** Find the number of employees in each department.
- Q75.** Find the average salary in each department.
- Q76.** Find the maximum salary in each department.
- Q77.** Find the minimum salary in each department.
- Q78.** Find the total salary in each department.
- Q79.** Find the number of employees in each city.
- Q80.** Find the average salary in each city.
- Q81.** Find departments having more than 5 employees.
- Q82.** Find departments whose average salary is greater than 60000.
- Q83.** Find cities whose total salary is greater than 200000.
- Q84.** Find the department with the highest total salary.
- Q85.** Find the department with the highest average salary.
- Q86.** Display employee count and average salary for each department and city combination.
- Q87.** Find the difference between the maximum and minimum salary in each department.
- Q88.** Find departments where the maximum salary is greater than 70000.
- Q89.** Find cities having at least 3 employees with salary above 60000.
- Q90.** Find the total number of distinct departments and distinct cities.

5. Constraints, Keys & Data Integrity (91–105)

- Q91.** Create a products table with product_id as PRIMARY KEY.
- Q92.** Create a users table where email must be UNIQUE.
- Q93.** Create an orders table where order_amount must be greater than 0 using CHECK.
- Q94.** Create a customers table where customer_name cannot be NULL.
- Q95.** Create a table with a DEFAULT status value of 'Active'.

- Q96.** Insert a row that uses the DEFAULT value.
- Q97.** Attempt to insert a duplicate PRIMARY KEY and explain which constraint is violated.
- Q98.** Attempt to insert a duplicate UNIQUE email and explain which constraint is violated.
- Q99.** Create employees and departments with a FOREIGN KEY relationship.
- Q100.** Insert valid parent and child records respecting the FOREIGN KEY.
- Q101.** Attempt to insert an employee with a department_id that does not exist.
- Q102.** Add a CHECK constraint to allow age only between 18 and 60.
- Q103.** Create a composite PRIMARY KEY using two columns.
- Q104.** Create an attendance table where employee_id and attendance_date together form a composite key.
- Q105.** Create a table with a named constraint and then identify its purpose.

6. JOINS Including Self Join (106–135)

- Q106.** Display employee_name and department_name using an INNER JOIN.
- Q107.** Display employee_name, salary, department_name, and location using a JOIN.
- Q108.** Find employees working in the Data Science department using a JOIN.
- Q109.** Find employees working in departments located in Hyderabad.
- Q110.** Display all employees even if they do not belong to any department.
- Q111.** Display all departments even if they have no employees.
- Q112.** Find departments that currently have no employees.
- Q113.** Find employees who do not have a matching department.
- Q114.** Find the number of employees in each department using JOIN and GROUP BY.
- Q115.** Find the average salary in each department using JOIN and GROUP BY.
- Q116.** Display each department with employee count including departments with zero employees.
- Q117.** Display each department with total salary including departments with zero employees.
- Q118.** Join departments with projects and display department_name and project_name.
- Q119.** Display employees, department names, and project names using appropriate joins.
- Q120.** Find departments that have at least one project.
- Q121.** Find departments that have no projects.
- Q122.** Create an employee-manager relationship using manager_id in the employees table.
- Q123.** Using a SELF JOIN, display employee name and manager name.
- Q124.** Using a SELF JOIN, display only employees who have managers.
- Q125.** Using a SELF JOIN, display employees who do not have managers.
- Q126.** Find all managers who manage at least one employee.
- Q127.** Count the number of employees reporting to each manager.
- Q128.** Find managers who manage more than 3 employees.
- Q129.** Display employee, manager, and employee salary using a SELF JOIN.

- Q130.** Find employees whose salary is greater than their manager's salary.
- Q131.** Find employees who work in the same department as their manager.
- Q132.** Use a LEFT JOIN to find unmatched rows between employees and departments.
- Q133.** Use multiple JOINS to display employee, department, and project information.
- Q134.** Find the total project budget for each department.
- Q135.** Find departments whose total project budget is greater than their department budget.

7. Subqueries (136–155)

- Q136.** Find employees whose salary is greater than the overall average salary.
- Q137.** Find the employee or employees with the highest salary using a subquery.
- Q138.** Find the employee or employees with the lowest salary using a subquery.
- Q139.** Find employees who earn the same salary as Alice.
- Q140.** Find employees who earn more than Alice.
- Q141.** Find employees working in the same department as Alice.
- Q142.** Find employees whose salary is greater than the average salary of their department.
- Q143.** Find employees whose salary is less than the average salary of their department.
- Q144.** Find the department with the highest average salary using a subquery.
- Q145.** Find departments whose average salary is greater than the company average salary.
- Q146.** Find employees who work in departments located in Hyderabad using a subquery.
- Q147.** Find employees who do not belong to any existing department using NOT IN or NOT EXISTS.
- Q148.** Find departments that have at least one employee using EXISTS.
- Q149.** Find departments that have no employees using NOT EXISTS.
- Q150.** Find employees whose salary is greater than every employee in the HR department.
- Q151.** Find employees whose salary is greater than at least one employee in the Finance department.
- Q152.** Find the second-highest distinct salary using a subquery.
- Q153.** Find the third-highest distinct salary using a subquery.
- Q154.** Find employees with the second-highest distinct salary.
- Q155.** Use a correlated subquery to find employees earning above their department average.

8. CTEs — Common Table Expressions (156–170)

- Q156.** Create a CTE containing employees whose salary is greater than 60000.
- Q157.** Use a CTE to calculate the company average salary and display employees earning above it.
- Q158.** Use a CTE to calculate average salary by department.
- Q159.** Use a CTE to find departments with average salary greater than 60000.
- Q160.** Create two CTEs: one for department statistics and one for high-paying departments.
- Q161.** Use a CTE to calculate total salary by city and filter cities above a chosen threshold.
- Q162.** Use a CTE to find the highest salary in each department.

Q163. Use a CTE to find employees earning above their department average.

Q164. Use a CTE to rank employees by salary.

Q165. Use a CTE to find the top 2 highest-paid employees.

Q166. Use a CTE with ROW_NUMBER to find the highest-paid employee in each department.

Q167. Use a CTE with DENSE_RANK to find the second-highest salary in each department.

Q168. Create a CTE that joins employees and departments, then filter the result.

Q169. Create a CTE containing departments with employee counts and filter departments with more than 3 employees.

Q170. Create multiple dependent CTEs where the second CTE uses the first CTE.

9. Window Functions (171–180)

Q171. Assign a unique row number to all employees ordered by salary descending.

Q172. Assign a row number to employees within each department ordered by salary descending.

Q173. Rank employees by salary using RANK().

Q174. Rank employees by salary using DENSE_RANK().

Q175. Compare ROW_NUMBER, RANK, and DENSE_RANK on duplicate salary values.

Q176. Display the company average salary beside every employee using AVG() OVER().

Q177. Display the department average salary beside every employee using PARTITION BY.

Q178. Calculate a running total of salary ordered by hire_date.

Q179. Use LAG() to display the previous employee's salary when ordered by hire_date.

Q180. Use LEAD() to display the next employee's salary when ordered by hire_date.

10. Hard SQL Challenge Questions (181–200)

Q181. Find the top 3 highest-paid employees in every department, including ties if appropriate. Hint: Compare ROW_NUMBER, RANK, and DENSE_RANK before choosing one.

Q182. Find the employee whose salary is closest to the average salary of their department. Hint: Calculate department average with a window function, then compare absolute differences.

Q183. For every department, find the difference between its highest-paid and second-highest distinct salary. Hint: Use DENSE_RANK in a CTE.

Q184. Find employees whose salary is higher than their manager and whose department average salary is also above the company average. Hint: Combine SELF JOIN, window/aggregate logic, and a CTE.

Q185. Find the first employee hired in each department and the most recently hired employee in each department. Hint: Use two rankings ordered in opposite directions.

Q186. Find departments where every employee earns more than 50000. Hint: Think about MIN(salary) or NOT EXISTS.

Q187. Find departments where at least 70 percent of employees earn above the company average salary. Hint: Use conditional aggregation and a company-average CTE.

Q188. Find the longest consecutive hiring-date gap between employees when ordered by hire_date. Hint: Use LAG() and date difference logic.

- Q189.** Find employees whose salary increased relative to the previous employee in their department when ordered by hire_date. Hint: PARTITION BY department with LAG().
- Q190.** Find the department with the largest salary inequality, measured as maximum salary minus minimum salary. Hint: Aggregate first, then rank the result.
- Q191.** Find employees who share their salary with another employee in the same department. Hint: GROUP BY department and salary, or use a window COUNT().
- Q192.** Find the third-highest distinct salary in each department and return all employees who earn it. Hint: DENSE_RANK plus a CTE.
- Q193.** Find departments whose total employee salary exceeds their project budget, and return the difference. Hint: Aggregate employee salary and project budget separately before joining.
- Q194.** Find employees who belong to departments with no projects but earn above their department average. Hint: Combine NOT EXISTS with a window calculation.
- Q195.** Return one row per department containing the employee with the highest salary, the employee with the lowest salary, and the department average. Hint: Use conditional aggregation after ranking.
- Q196.** Find employees who are in the top 20 percent of salaries within their department. Hint: Investigate NTILE(5) or percentile-related window logic supported by your SQL database.
- Q197.** Find the median salary in each department. Hint: The exact syntax depends on your SQL database; consider ROW_NUMBER plus counts if percentile functions are unavailable.
- Q198.** Find employees whose salary is greater than the average salary of all employees hired before them. Hint: This requires a running window frame or correlated logic.
- Q199.** Find managers whose direct reports collectively earn more than the manager and identify the difference. Hint: SELF JOIN, GROUP BY, and aggregation.
- Q200.** Create a report showing for every employee: company salary rank, department salary rank, department average, difference from department average, previous salary by hire date, and next salary by hire date. Hint: Use multiple window functions in one SELECT or organize them with a CTE.